Track 1 RibEye: Project Profile Document (PDF)

Project background

Radiologists face increasing workloads with rising demand for medical imaging analysis. Chest X-ray interpretation requires specialized expertise, is time-consuming, and is prone to human error under high-volume conditions. Additionally, AI "black box" systems provide predictions without explanation, reducing clinical trust and adoption.

Our **Explainable AI Radiology Assistant** is a multimodal AI system that combines computer vision, natural language generation, and interactive chat to assist radiologists in analyzing chest X-rays. Unlike conventional AI diagnostic tools, our setup follows a neuro-symbolic approach by providing the LLM with structured findings (e.g., cardiomegaly, pleural effusion, pneumonia) predicted by a classifier pre-trained on chest X-ray classification. Furthermore, our system provides full transparency by linking every report statement to the corresponding image region through explainable heatmaps.

Since many doctors' offices are inherently constrained by their computational demand, and hosting medical software on the cloud leaks privacy, we opt for a lightweight approach that’s capable of running on low-end consumer GPUs. Specifically, we utilize a model with ternary quantization. GPUs benefit from ternary quantization due to their small model size and due to an algorithmic advantage. The small model size allows efficient parameter transfer in the parameter hierarchy of the GPU cores. Since parameters are one of three values -1, 0, or 1, each computation boils down to only additions, instead of more expensive multiplications.

Target users & application scenarios

Our application analyzes chest X-ray scans and generates reports based on the results, mainly to support radiologists. As they analyze the X-ray scans on their own, getting an additional opinion from an AI assistant might reduce the error rate. By providing information such as the different pathology classes and confidence scores for each finding, it might help the radiologist with uncertain cases that might be undiscovered or mislabeled without the additional input. In case of doubt, the radiologist can also discuss the findings with the model via the chat interface.

While our application mainly supports radiologists, it can also serve other medical personnel in rapid triage cases, as in emergency wards. Often, due to the radiologist’s timetable, the procedure is stalled waiting for the doctor’s diagnosis. However, with the support of our application, an ad hoc secondary diagnosis from

the model can help the personnel prioritize which patients need the most urgent attention while awaiting the radiologist's confirmed diagnosis.

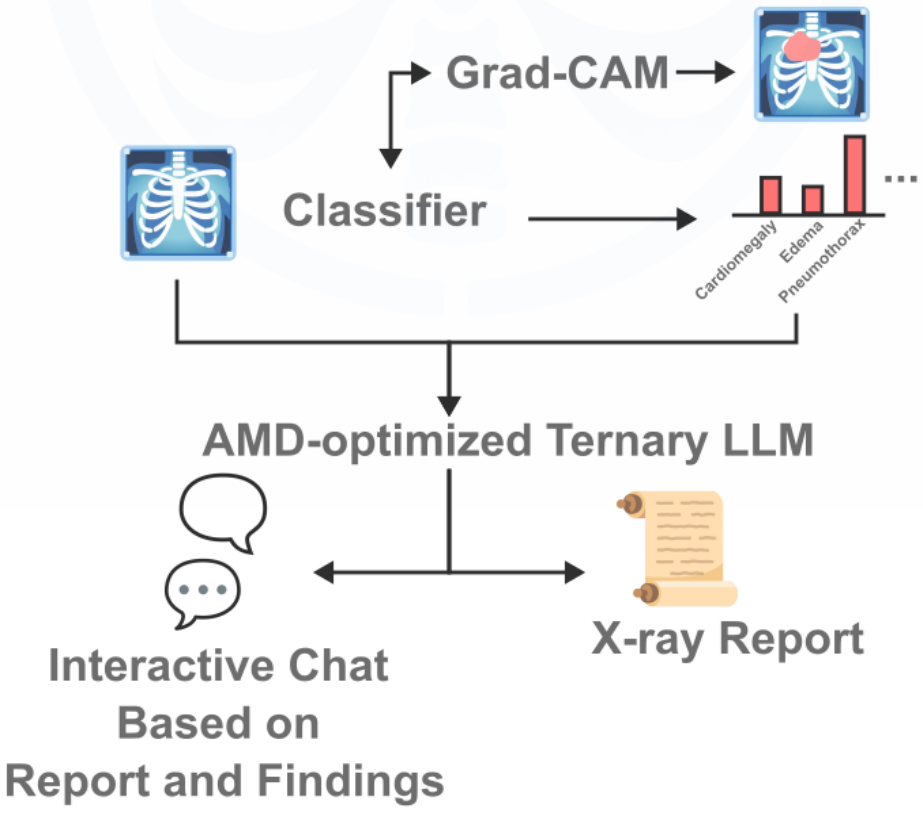
Due to the usage of ternary models, our application fits on a single consumer AMD Radeon GPU requiring roughly 8GB VRAM. This allows for widespread deployment, since the computational requirements are relatively low and thus, the hardware is cheaper.

System architecture

The system utilizes three different models. The torchxrayvision DenseNet and two Ternary Bonsai models, namely one model and its respective vision encoder. The Ternary Bonsai models are run on a llama.cpp instance, enabling LLM inference with minimal setup and state-of-the-art performance.

After an X-ray torso scan has been uploaded via the React frontend, it gets passed to the torchxrayvision DenseNet for classification.

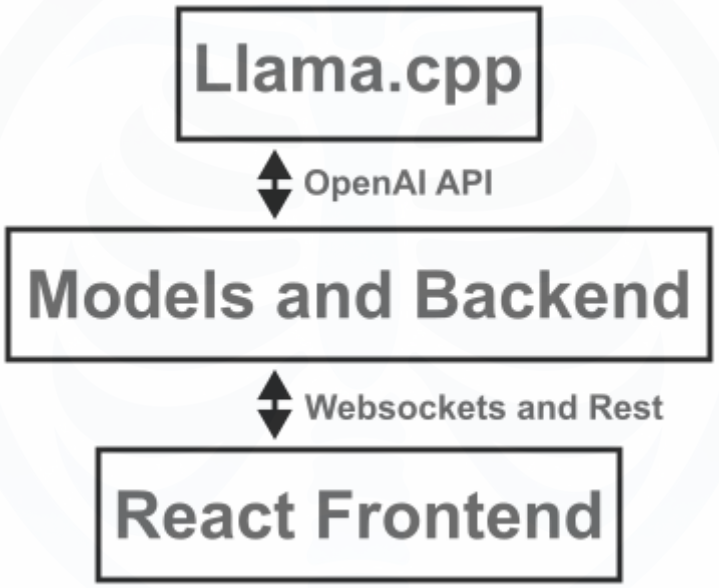
The classification results are then processed via Grad-CAM to generate the segmentation mask before the results are passed to the Ternary Bonsai models. Based on the results, they generate a report. The user receives a report that they can download. Upon interacting with the LLM via a chat interface, the user can ask the LLM for further information. The chat history can also be found in the report.



Our application provides an intuitive React frontend for the end-user. Via HTTP, WebSockets, and REST (built via FastAPI), the frontend communicates with the backend, which also contains the models. The backend communicates with the Llama server containing the LLM via the OpenAI API.

Within the application, there is no server-side session state. Each request is fully self-contained. The frontend sends the image, findings, and chat history it already has. Thus, the backend does not have to store anything between requests. This architecture enables safe concurrent sessions with no server-side data mixing, preventing server-side data leakage. By also introducing asynchronous functionality, this allows for efficient streamlining while waiting for other processes to execute.

Since our backend is stateless-per-request, horizontal scaling does not require solving session-affinity or shared-session-store problems, making our application scalable in the future without requiring the application logic itself to be redesigned.



Model & Algorithm Introduction

Our application utilizes several models and algorithms which we are going to introduce here.

Classifier: The TorchXRayVision classifier is a DenseNet121 (CNN) pre-trained on the large-scale X-ray dataset CheXpert. The input images are resized to a resolution of 224x224. The model is a multi-class predictor, meaning the model predicts a separate confidence score (0% - 100%) for each potential finding.

Grad-CAM: The Grad-CAM (Gradient-weighted Class Activation Mapping) algorithm is a lightweight technique for explaining predictions of a CNN. Essentially, the algorithm computes the absolute gradient of the last convolutional layer's activations

with respect to the final prediction. The upscaled gradient map serves as the explanation for the respective prediction. Grad-CAM is applied post-hoc, without requiring adaptations to the model.

LLM: We use the Ternary Bonsai 27B LLM, which is a heavily quantized version of Qwen3.6-27B. The ternary model is quantized by Prism ML using a proprietary quantization mechanism. The model retains a 4-bit KV cache, which contains the input image, the generated report, and the previous chat history. The LLM offers vision capabilities via a CLIP-based vision encoder quantized to 8-bit.

Adaptation description for AMD Radeon GPU / ROCm

The chat- and report-generation model is served via a fork of llama.cpp maintaining a custom ternary quantization kernel format. This runtime required a backend-level adaptation for AMD.

While we initially ran the build on Vulkan, which utilizes the AMD GPUs via the open-source RADV driver, we later switched to an official ROCm/HIP backend, which included pre-built ROCm release binaries. Thus, the LLM inference path runs on ROCm/HIP rather than Vulkan.

This can be verified during startup time. Our logs report the active devices, such as ROCm: AMD Radeon Graphics, with ROCm-specific runtime flags (NO_VMM). amd-smi process lists the running server as a tracked GPU process, which was not the case under Vulkan.

We loaded the torchxrayvision DenseNet, our chest X-ray classifier, through plain PyTorch. No CUDA-specific API is used anywhere in this model. Further, the ROCm build exposes the same cuda namespace as its CUDA build with no separate rocm device type. Thus, CUDA calls run unmodified on the AMD hardware once a ROCm build of torch is installed in place of the default CUDA wheel. The adaptation was environmental rather than architectural. By installing the ROCm-matched torch build and adding explicit handling, such that the DEVICE=rocm setting resolves correctly to the CUDA string PyTorch itself expects.

The ternary Bonsai models require the custom kernel in the llama fork. The model weights themselves are backend-agnostic. Its behavior is the same on CUDA, Vulkan, and ROCm/HIP builds. Only the compute kernels operating on those bytes differ.